

EECS 3311 Fall 2026 Course Project

Stage 1 Project Design Report

VoyageAI - Intelligent Travel Planning Agent

Replace with: Name | Student Number | Section | Submission Date

1. Project Overview

1.1 Problem and Motivation

Planning a trip often requires a traveler to search multiple sources for attractions, restaurants, accommodation, transportation, weather, and prices, then manually combine the results into a workable schedule. The process becomes harder when the traveler has a strict budget, dietary restrictions, preferred activities, limited time, transportation preferences, or weather-sensitive plans.

VoyageAI addresses this problem with a single AI-agent-based travel planning system. The agent interprets natural-language requests, gathers information through travel tools, reasons about user constraints, creates multi-day plans, checks budget and schedule feasibility, remembers useful preferences, and revises plans when conditions or user requests change.

1.2 Target Users

The primary users are travelers who want structured assistance planning personal trips, including students, solo travelers, families, tourists visiting unfamiliar destinations, and budget-conscious travelers.

1.3 Why an AI Agent Is Appropriate

Travel planning is not a single deterministic calculation. A useful planner must understand natural language, reason over multiple constraints, decide what information to retrieve, call external tools, combine results, remember context, and execute multi-step workflows. VoyageAI therefore uses an agent architecture rather than a simple GUI that forwards a prompt directly to an LLM.

1.4 Planned AI/LLM Model

The planned implementation will use an OpenAI GPT-family model through the LLMClient abstraction. The implementation class is OpenAILLMClient. The exact configured model name is isolated from the rest of the application so the model can be changed without redesigning the system.

1.5 AI Interaction with the Software

The GUI and CLI do not call the LLM directly. Both interact with TravelAgentController. The controller acts as a facade over the agent and domain services. TravelAgent coordinates TripPlanner, ToolManager, MemoryManager, and LLMClient. Deterministic services such as BudgetManager and ScheduleManager validate agent-generated plans before they become application state.

1.6 GUI and CLI

The GUI is a trip-planning dashboard with a natural-language request area, itinerary timeline, attraction/restaurant/accommodation panels, transportation and weather information, a budget view, and trip state/history areas. The CLI exposes the same major functionality through commands such as plan-trip, research-destination, recommend-attractions, recommend-restaurants, find-accommodation, plan-transportation, show-budget, check-conflicts, check-weather, modify-trip, alternative-plan, and packing-list.

1.7 High-Level Architecture

Traveler -> GUI/CLI -> TravelAgentController -> TravelAgent/Domain Services -> ToolManager/LLMClient -> External Services -> validated Trip/Itinerary -> GUI/CLI

2. Detailed Feature Specifications

F01 - User Travel Profile and Preferences

Description: Stores reusable travel preferences such as budget level, interests, dietary needs, accommodation preference, transportation preference, and desired travel pace.

User Interaction: Travel Profile screen in the GUI; corresponding profile commands in the CLI.

Input: Preference values entered or modified by the traveler.

Output: Validated and stored TravelProfile available to the agent.

AI Involvement: Hybrid

Expected Workflow: The GUI/CLI sends the profile to TravelAgentController. TravelProfileManager validates it, MemoryManager stores reusable preferences, and TripSession notifies the active view.

Error/Alternative Cases: Invalid or unsupported values are rejected with a clear correction message.

F02 - Natural-Language Trip Request

Description: Converts a free-form request into a structured TripRequest containing destination, dates, budget, interests, and constraints.

User Interaction: AI planning panel in the GUI or plan-trip command in the CLI.

Input: Natural-language trip request.

Output: Structured TripRequest or a clarification request.

AI Involvement: Hybrid

Expected Workflow: TravelAgentController forwards the text to TravelAgent, which uses LLMClient.extractTripRequest(). Required fields are validated before planning continues.

Error/Alternative Cases: Missing or ambiguous essential information causes a clarification request instead of silent guessing.

F03 - Destination Research

Description: Retrieves and organizes useful destination information before or during planning.

User Interaction: Research Destination action or natural-language request.

Input: Destination and optional research interests.

Output: Structured destination research summary.

AI Involvement: Hybrid

Expected Workflow: TravelAgent uses ToolManager and DestinationSearchTool, then LLMClient organizes the retrieved information into a concise result.

Error/Alternative Cases: If retrieval fails, the system reports the failure and does not present unsupported data as retrieved facts.

F04 - Multi-Day Itinerary Generation

Description: Creates a day-by-day itinerary that satisfies travel dates, preferences, budget, and scheduling constraints.

User Interaction: Generate Itinerary button or plan-trip CLI flow.

Input: TripRequest, TravelProfile, retrieved context, budget limit.

Output: Trip containing an Itinerary and BudgetSummary.

AI Involvement: Hybrid

Expected Workflow: TripPlanner uses a PlanningStrategy, travel tools, LLMClient, ScheduleManager, and BudgetManager to generate and validate the plan.

Error/Alternative Cases: If the first plan violates important constraints, the planner revises it or explains which constraints cannot all be satisfied.

F05 - Attraction Recommendations

Description: Recommends attractions that fit the destination, interests, schedule, and budget.

User Interaction: Attractions panel or recommendation command.

Input: Destination, interests, dates, budget, current itinerary.

Output: Organized attraction recommendations.

AI Involvement: Hybrid

Expected Workflow: ToolManager invokes AttractionSearchTool; the AI ranks and explains candidates using the current profile and trip context.

Error/Alternative Cases: If no candidates satisfy restrictions, the system explains the constraint conflict and may suggest relaxing one.

F06 - Restaurant Recommendations

Description: Recommends restaurants considering cuisine preferences, dietary restrictions, budget, and itinerary location.

User Interaction: Restaurants panel or restaurant recommendation command.

Input: Location, cuisine, dietary restrictions, budget, timing.

Output: Suitable restaurant recommendations.

AI Involvement: Hybrid

Expected Workflow: RestaurantSearchTool retrieves candidates; deterministic filtering removes incompatible results; the AI ranks and explains remaining choices.

Error/Alternative Cases: If current restaurant data cannot be retrieved, the system clearly reports that current recommendations are unavailable.

F07 - Accommodation Planning

Description: Finds accommodation options that align with travel dates, area, budget, and lodging preferences.

User Interaction: Accommodation panel or accommodation command.

Input: Destination, dates, preferred area/type, budget.

Output: Accommodation options with estimated cost information.

AI Involvement: Hybrid

Expected Workflow: AccommodationSearchTool retrieves candidates; the agent evaluates them against profile and itinerary constraints.

Error/Alternative Cases: If no suitable option is found, the system explains which constraints prevented a match.

F08 - Transportation Planning

Description: Suggests transportation between itinerary locations using available travel data and user preferences.

User Interaction: Transportation panel, itinerary segment action, or CLI command.

Input: Origin, destination, time, preferences, itinerary context.

Output: Suggested transport option(s) and estimated travel information.

AI Involvement: Hybrid

Expected Workflow: ToolManager invokes TransportationTool; the agent evaluates available options in the context of the itinerary.

Error/Alternative Cases: If live/current data are unavailable, the system reports the limitation rather than inventing schedules.

F09 - Budget Estimation and Tracking

Description: Calculates estimated trip cost by category, compares it to the user's limit, and identifies over-budget plans.

User Interaction: Budget dashboard or show-budget command.

Input: Estimated costs from itinerary items and maximum budget.

Output: BudgetSummary, remaining budget, and warnings.

AI Involvement: Hybrid

Expected Workflow: BudgetManager performs deterministic calculations. If needed, the AI can propose lower-cost alternatives.

Error/Alternative Cases: Missing prices are marked as unknown/estimated and are never silently treated as zero.

F10 - Schedule Conflict Detection

Description: Detects overlapping activities and infeasible timing, including insufficient travel time between activities.

User Interaction: Automatic validation plus Check Schedule action/command.

Input: Activities, start times, durations, and travel-time information.

Output: Conflict list and optional resolution suggestions.

AI Involvement: Hybrid

Expected Workflow: ScheduleManager performs deterministic conflict checks; the agent may suggest resolutions.

Error/Alternative Cases: If timing data are incomplete, the system marks the schedule as incomplete rather than conflict-free.

F11 - Weather-Aware Itinerary Adjustment

Description: Uses weather data to identify unsuitable activities and revise the itinerary while preserving user constraints.

User Interaction: Weather Check / Adjust for Weather action or CLI command.

Input: Trip dates, destination, current itinerary, weather data.

Output: Revised weather-aware itinerary.

AI Involvement: Hybrid

Expected Workflow: WeatherTool retrieves weather information; TripPlanner revises affected activities; ScheduleManager and BudgetManager revalidate the result.

Error/Alternative Cases: If weather data are unavailable, the original itinerary is retained and the limitation is shown.

F12 - Natural-Language Itinerary Editing

Description: Lets the traveler modify the current itinerary using instructions such as moving, replacing, adding, or removing activities.

User Interaction: Agent chat/edit box or modify-trip CLI command.

Input: Current itinerary and natural-language edit instruction.

Output: Validated modified itinerary.

AI Involvement: Hybrid

Expected Workflow: TravelAgent interprets the instruction, ItineraryCommandFactory creates a concrete command, the command executes, and schedule/budget checks run again.

Error/Alternative Cases: Ambiguous edits trigger clarification; invalid edits are rejected without corrupting the current itinerary.

F13 - Alternative Plan Generation

Description: Creates a second itinerary with different priorities, pace, activities, or cost while preserving required constraints.

User Interaction: Generate Alternative action or alternative-plan command.

Input: Current trip and alternative-plan criteria.

Output: Alternative itinerary shown separately from the original.

AI Involvement: Hybrid

Expected Workflow: TripPlanner selects an appropriate PlanningStrategy, retrieves additional information if needed, and validates the alternative.

Error/Alternative Cases: If a meaningful alternative cannot satisfy all fixed constraints, the system identifies the limiting constraints.

F14 - Packing List Generation

Description: Generates a categorized packing list using destination, trip duration, activities, and expected weather.

User Interaction: Generate Packing List action or packing-list command.

Input: Trip, itinerary activities, dates, destination, weather when available.

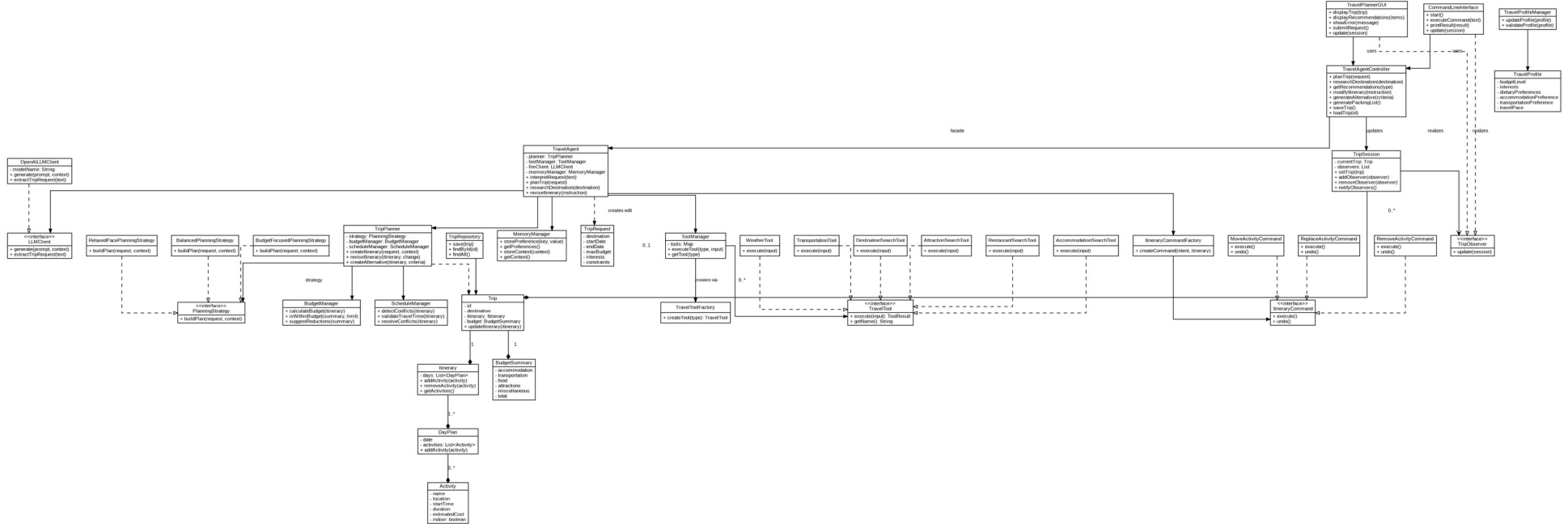
Output: Categorized packing list.

AI Involvement: Hybrid

Expected Workflow: The system retrieves weather when available and LLMClient generates a packing list using trip context.

Error/Alternative Cases: If weather retrieval fails, a general list may still be produced but is clearly labeled as not weather-specific.

3. UML Class Diagram



3.1 Major Class Responsibilities

Class / Interface	Responsibility
TravelPlannerGUI / CommandLineInterface	Boundary/view objects. Collect user input and present results; neither contains core planning logic.
TravelAgentController	Application facade/controller. Exposes use-case-level operations to GUI and CLI.
TravelAgent	Coordinates AI-specific work: request interpretation, tool-supported research, planning context, and natural-language revision.
TripPlanner	Builds and revises itineraries while delegating planning policy to PlanningStrategy.
PlanningStrategy	Strategy abstraction for different planning priorities.
ToolManager / TravelToolFactory / TravelTool	Tool subsystem that creates, selects, and executes retrieval/service tools.
LLMClient / OpenAILLMClient	Abstraction and concrete adapter-like implementation for the selected LLM service.
MemoryManager	Stores relevant reusable preferences and current agent context.
BudgetManager	Performs deterministic cost calculations and budget validation.
ScheduleManager	Performs deterministic time-overlap and travel-time validation.
ItineraryCommand hierarchy	Represents controlled itinerary changes as executable operations.
TripSession / TripObserver	Maintains current trip state and informs active views when it changes.
TripRepository	Persistence boundary for saving and loading trips.
Trip / Itinerary / DayPlan / Activity / BudgetSummary / TravelProfile / TripRequest	Core domain model.

4. Design Pattern Explanations

4.1 MVC

Design problem addressed: Separates presentation from application coordination and domain state.

Participating classes and roles: TravelPlannerGUI and CommandLineInterface are Views; TravelAgentController is the Controller; Trip, Itinerary, TravelProfile, TripSession and related services form the Model.

Why appropriate: The GUI/CLI should not contain planning, AI, persistence, budget, or scheduling logic. MVC keeps presentation replaceable and lets both interfaces reuse the same core system.

Without the pattern: Without MVC, GUI/CLI code would duplicate business logic and changes to one interface could affect the agent/domain implementation.

4.2 Facade

Design problem addressed: Provides one simplified application entry point to a complex agent subsystem.

Participating classes and roles: TravelAgentController is the facade. It coordinates TravelAgent, TripPlanner, ToolManager, MemoryManager, BudgetManager, ScheduleManager and repositories.

Why appropriate: The GUI and CLI need a small stable API even though the underlying agent workflow uses many components.

Without the pattern: Without the facade, views would depend directly on many internal services, increasing coupling and making the interface harder to maintain.

4.3 Strategy

Design problem addressed: Allows the planner to change how itineraries are produced without changing TripPlanner.

Participating classes and roles: PlanningStrategy defines buildPlan(). BalancedPlanningStrategy, BudgetFocusedPlanningStrategy and RelaxedPacePlanningStrategy are concrete strategies. TripPlanner is the context.

Why appropriate: Different users can prioritize balance, cost, or slower pace. The planning policy can be selected at runtime from profile/request data.

Without the pattern: Without Strategy, TripPlanner would accumulate large conditional blocks for every planning style and would be harder to extend or test.

4.4 Factory Method

Design problem addressed: Centralizes creation of travel tools and hides concrete tool classes from their clients.

Participating classes and roles: TravelToolFactory creates DestinationSearchTool, AttractionSearchTool, RestaurantSearchTool, AccommodationSearchTool, WeatherTool and TransportationTool, all implementing TravelTool.

Why appropriate: The agent needs multiple tools and should not construct each concrete tool directly.

Without the pattern: Without a factory, ToolManager/agent code would be tightly coupled to constructors and adding new tools would require changes throughout the system.

4.5 Command

Design problem addressed: Represents itinerary edits as executable objects and supports structured modification/undo.

Participating classes and roles: ItineraryCommand is the command interface. MoveActivityCommand, ReplaceActivityCommand and RemoveActivityCommand are concrete commands. ItineraryCommandFactory creates commands from parsed edit intents.

Why appropriate: Natural-language edits should become controlled domain operations rather than allowing the LLM to directly mutate the itinerary.

Without the pattern: Without Command, edit logic would be scattered through the agent/controller, validation would be harder, and undo/auditing would be difficult.

4.6 Observer

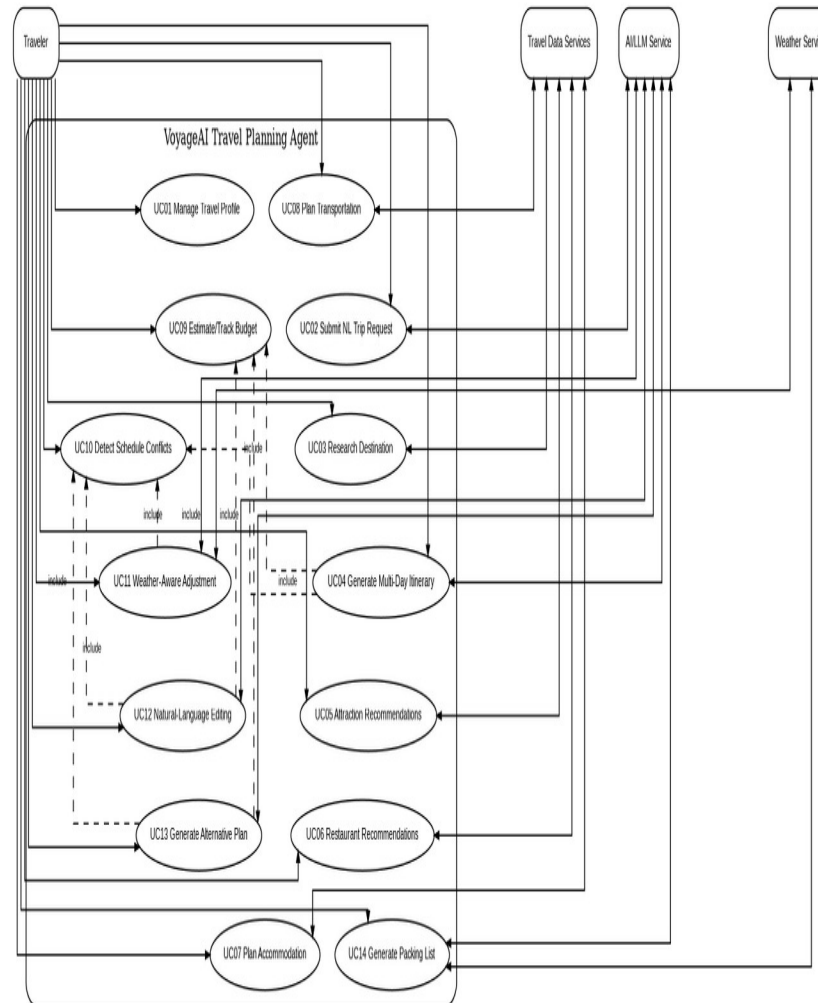
Design problem addressed: Keeps active views synchronized when trip state changes.

Participating classes and roles: TripSession is the subject. TripObserver is the observer interface. TravelPlannerGUI and CommandLineInterface implement/update from it.

Why appropriate: Many operations change the same trip state: planning, editing, weather revision, budget updates. Views should refresh without the domain depending on a specific GUI component.

Without the pattern: Without Observer, every service would need direct knowledge of interface elements or the controller would require repetitive manual refresh calls.

5. UML Use-Case Diagram



The primary actor is the Traveler. External actors/services include the AI/LLM service, travel-data services, and a weather service. The use cases collectively cover all 14 major features.

6. Detailed Use-Case Descriptions

UC01 - Manage Travel Profile

Actor(s): Traveler

Goal: Create or update reusable travel preferences.

Preconditions: Application is running.

Trigger: Traveler opens profile management.

Main Success Scenario: 1. System displays current profile. 2. Traveler enters or edits preferences. 3. System validates values. 4. Profile is stored. 5. Memory is updated. 6. View confirms success.

Alternative/Exception Flows: A1. Invalid value: system identifies the field and requests correction. A2. Storage failure: existing profile remains unchanged and an error is shown.

Postconditions: A valid TravelProfile is available to future agent operations.

Related Feature(s): F01

UC02 - Submit Natural-Language Trip Request

Actor(s): Traveler, AI/LLM Service

Goal: Convert a free-form travel request into structured planning constraints.

Preconditions: Application is running.

Trigger: Traveler submits a natural-language trip request.

Main Success Scenario: 1. Controller receives the text. 2. TravelAgent sends it to LLMClient for structured extraction. 3. A TripRequest is created. 4. Required fields are validated. 5. Structured request is displayed/accepted for planning.

Alternative/Exception Flows: A1. Missing destination/dates: system asks for clarification. A2. LLM output cannot be parsed: system retries safely or asks for structured input.

Postconditions: A valid TripRequest exists or a clarification has been requested.

Related Feature(s): F02

UC03 - Research Destination

Actor(s): Traveler, Travel Data Services, AI/LLM Service

Goal: Retrieve and summarize information about the chosen destination.

Preconditions: A destination is known.

Trigger: Traveler requests destination research.

Main Success Scenario: 1. Agent determines research need. 2. ToolManager gets DestinationSearchTool. 3. Tool retrieves destination information. 4. LLM organizes relevant results. 5. System displays the research summary.

Alternative/Exception Flows: A1. Tool unavailable: report retrieval failure. A2. Empty results: state that no matching information was returned.

Postconditions: Retrieved information is available as planning context.

Related Feature(s): F03

UC04 - Generate Multi-Day Itinerary

Actor(s): Traveler, AI/LLM Service, Travel Data Services

Goal: Create a complete validated day-by-day itinerary.

Preconditions: A valid TripRequest exists.

Trigger: Traveler selects Generate Itinerary.

Main Success Scenario: 1. Controller calls TravelAgent.planTrip(). 2. TripPlanner selects PlanningStrategy. 3. Required tools retrieve context. 4. LLM helps construct a proposed itinerary. 5. ScheduleManager detects conflicts. 6. BudgetManager calculates cost. 7. Planner revises if required. 8. TripSession stores the resulting trip. 9. View displays it.

Alternative/Exception Flows: A1. Required data unavailable: continue only with clearly marked limitations or ask user. A2. Budget/schedule conflict cannot be resolved: explain unsatisfied constraint. A3. AI response invalid: retry/validate before accepting.

Postconditions: A Trip with a validated Itinerary and BudgetSummary is available.

Related Feature(s): F04

UC05 - Get Attraction Recommendations

Actor(s): Traveler, Travel Data Services, AI/LLM Service

Goal: Find attractions suitable for the current traveler and trip.

Preconditions: Destination is known.

Trigger: Traveler requests attraction recommendations.

Main Success Scenario: 1. ToolManager invokes AttractionSearchTool. 2. Candidates are retrieved. 3. Agent evaluates profile, schedule and budget. 4. LLM produces explanations. 5. Recommendations are displayed.

Alternative/Exception Flows: A1. No matches: explain why and offer constraint relaxation. A2. Retrieval error: report unavailable current data.

Postconditions: Recommendation set is displayed and may be added to itinerary.

Related Feature(s): F05

UC06 - Get Restaurant Recommendations

Actor(s): Traveler, Travel Data Services, AI/LLM Service

Goal: Find restaurants compatible with cuisine, dietary, location and budget constraints.

Preconditions: Destination/current area is known.

Trigger: Traveler requests restaurants.

Main Success Scenario: 1. RestaurantSearchTool retrieves candidates. 2. System removes incompatible options. 3. Agent ranks remaining choices. 4. System displays recommendations and rationale.

Alternative/Exception Flows: A1. No dietary-compatible matches: report no match. A2. Service failure: report current restaurant data unavailable.

Postconditions: Restaurant recommendations are available to the traveler.

Related Feature(s): F06

UC07 - Plan Accommodation

Actor(s): Traveler, Travel Data Services, AI/LLM Service

Goal: Identify suitable accommodation options.

Preconditions: Destination and dates are known.

Trigger: Traveler opens accommodation planning.

Main Success Scenario: 1. Validate dates/preferences. 2. AccommodationSearchTool retrieves candidates. 3. Agent compares candidates with budget/location preferences. 4. Suitable options are displayed.

Alternative/Exception Flows: A1. No suitable options: identify conflicting constraints. A2. Service error: display retrieval failure.

Postconditions: Selected/recommended accommodation can be associated with the trip.

Related Feature(s): F07

UC08 - Plan Transportation

Actor(s): Traveler, Travel Data Services, AI/LLM Service

Goal: Suggest transport between itinerary locations.

Preconditions: Trip contains at least two locations or origin/destination are supplied.

Trigger: Traveler requests transportation planning.

Main Success Scenario: 1. TransportationTool retrieves options. 2. Agent evaluates timing, preference and cost. 3. Suitable options are shown. 4. Selected estimate may be added to itinerary/budget.

Alternative/Exception Flows: A1. No route data: report unavailable. A2. Option conflicts with schedule: flag conflict and suggest alternative.

Postconditions: Transportation recommendation is available and may affect budget/schedule.

Related Feature(s): F08

UC09 - Estimate and Track Budget

Actor(s): Traveler

Goal: Calculate trip cost and compare it with the maximum budget.

Preconditions: Trip contains one or more estimated costs.

Trigger: Traveler opens budget view or a trip change triggers recalculation.

Main Success Scenario: 1. BudgetManager collects costs. 2. Totals are calculated by category. 3. Total is compared with limit. 4. BudgetSummary is displayed. 5. If over budget, agent may suggest reductions.

Alternative/Exception Flows: A1. Missing cost: mark unknown/estimated. A2. Invalid negative cost: reject invalid data.

Postconditions: Current BudgetSummary is stored with the trip.

Related Feature(s): F09

UC10 - Detect Schedule Conflicts

Actor(s): Traveler

Goal: Identify overlapping or infeasible itinerary activities.

Preconditions: An itinerary exists.

Trigger: Traveler requests a schedule check or itinerary changes.

Main Success Scenario: 1. ScheduleManager orders activities. 2. It checks overlaps. 3. It checks required travel time when available. 4. Conflicts are returned. 5. Agent may suggest resolutions.

Alternative/Exception Flows: A1. Missing timing information: mark schedule incomplete. A2. Travel time unavailable: conflict confidence is limited and shown.

Postconditions: Conflict status is attached to the current itinerary.

Related Feature(s): F10

UC11 - Adjust Itinerary for Weather

Actor(s): Traveler, Weather Service, AI/LLM Service

Goal: Revise activities affected by expected weather.

Preconditions: A trip and dates exist.

Trigger: Traveler requests weather-aware adjustment.

Main Success Scenario: 1. WeatherTool retrieves forecast/context. 2. Agent identifies weather-sensitive activities. 3. TripPlanner proposes replacements/moves. 4. ScheduleManager rechecks conflicts. 5. BudgetManager recalculates cost. 6. Revised itinerary is displayed.

Alternative/Exception Flows: A1. Weather unavailable: keep original and show limitation. A2. No suitable replacements: explain affected activities and leave them unchanged unless user decides otherwise.

Postconditions: A validated revised itinerary is available or the original remains unchanged.

Related Feature(s): F11

UC12 - Edit Itinerary with Natural Language

Actor(s): Traveler, AI/LLM Service

Goal: Apply controlled itinerary changes from natural language.

Preconditions: An itinerary exists.

Trigger: Traveler enters an editing instruction.

Main Success Scenario: 1. LLM interprets edit intent. 2. ItineraryCommandFactory creates a concrete command. 3. Command executes. 4. Schedule and budget are revalidated. 5. TripSession is updated. 6. View refreshes.

Alternative/Exception Flows: A1. Ambiguous edit: ask clarification. A2. Invalid target activity: report not found. A3. Edit creates unresolved conflict: reject or present resolution options.

Postconditions: A validated modified itinerary is stored; previous state remains recoverable through command undo/history design.

Related Feature(s): F12

UC13 - Generate Alternative Plan

Actor(s): Traveler, AI/LLM Service, Travel Data Services

Goal: Create a different itinerary while preserving fixed constraints.

Preconditions: A current trip exists.

Trigger: Traveler requests an alternative with criteria.

Main Success Scenario: 1. Planner derives change criteria. 2. Appropriate PlanningStrategy is selected. 3. Additional data are retrieved if needed. 4. Alternative is generated. 5. Schedule and budget checks run. 6. Alternative is shown separately.

Alternative/Exception Flows: A1. No feasible alternative: explain limiting constraints. A2. Retrieval failure: use existing context only if sufficient and disclose limitation.

Postconditions: A validated alternative itinerary exists without replacing the original automatically.

Related Feature(s): F13

UC14 - Generate Packing List

Actor(s): Traveler, Weather Service, AI/LLM Service

Goal: Produce a practical packing list tailored to trip context.

Preconditions: A trip exists.

Trigger: Traveler selects Generate Packing List.

Main Success Scenario: 1. System gathers destination, duration and activities. 2. Weather is retrieved when available. 3. LLM generates categorized items. 4. Result is displayed.

Alternative/Exception Flows: A1. Weather unavailable: generate a general list and label limitation. A2. Trip lacks enough details: request missing context.

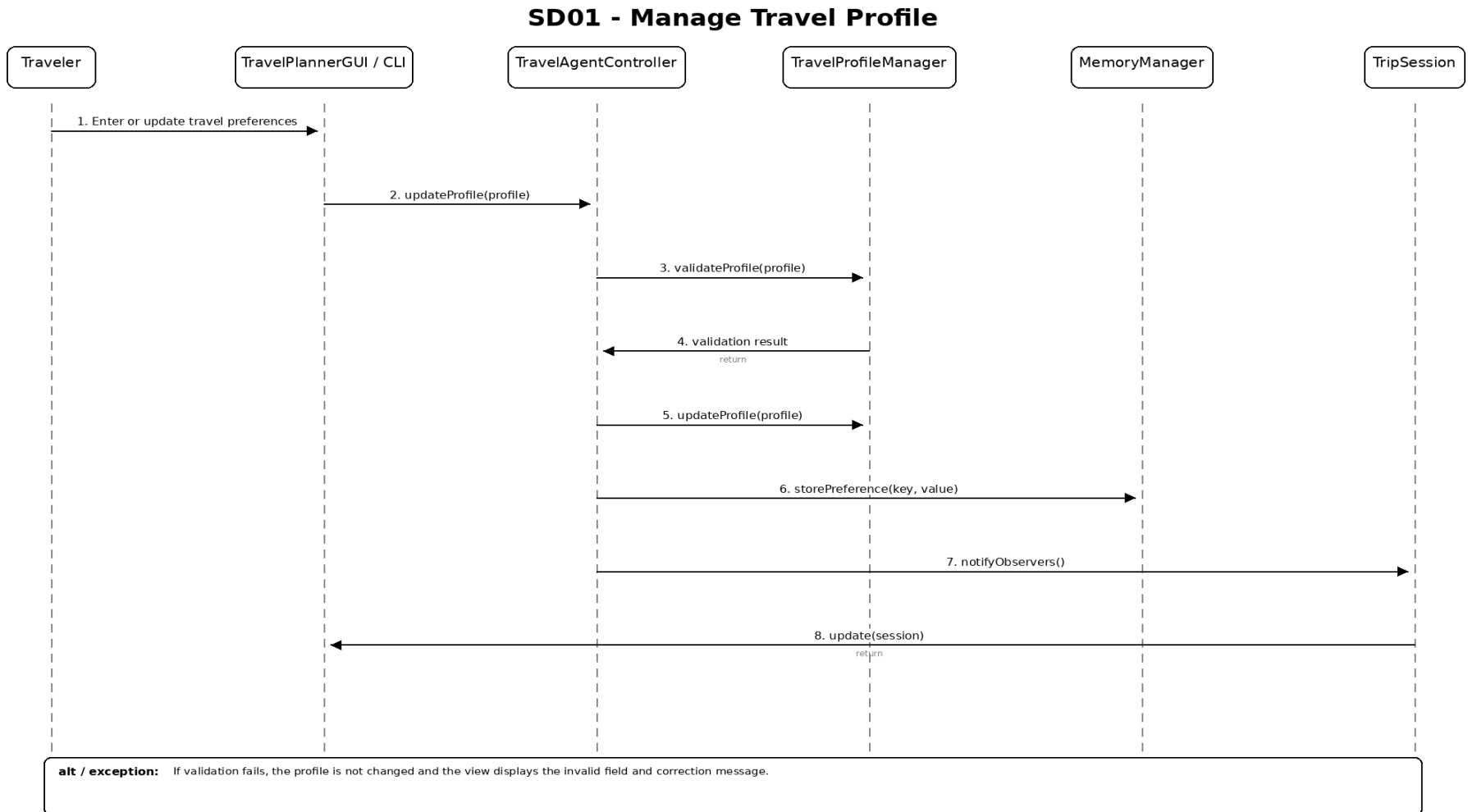
Postconditions: A categorized packing list is available to the traveler.

Related Feature(s): F14

7. Sequence Diagrams

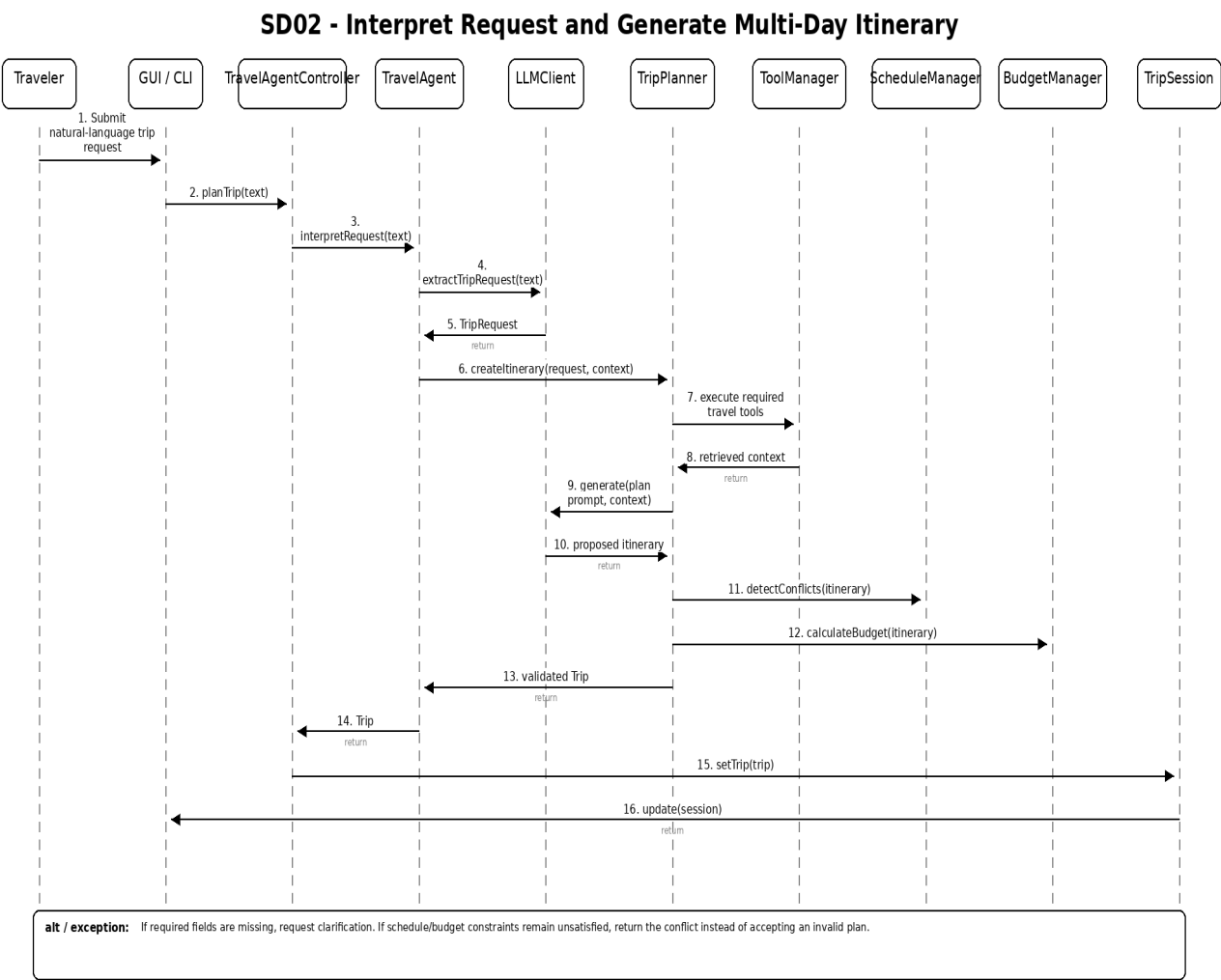
SD01 - Manage Travel Profile

Covered Features: F01



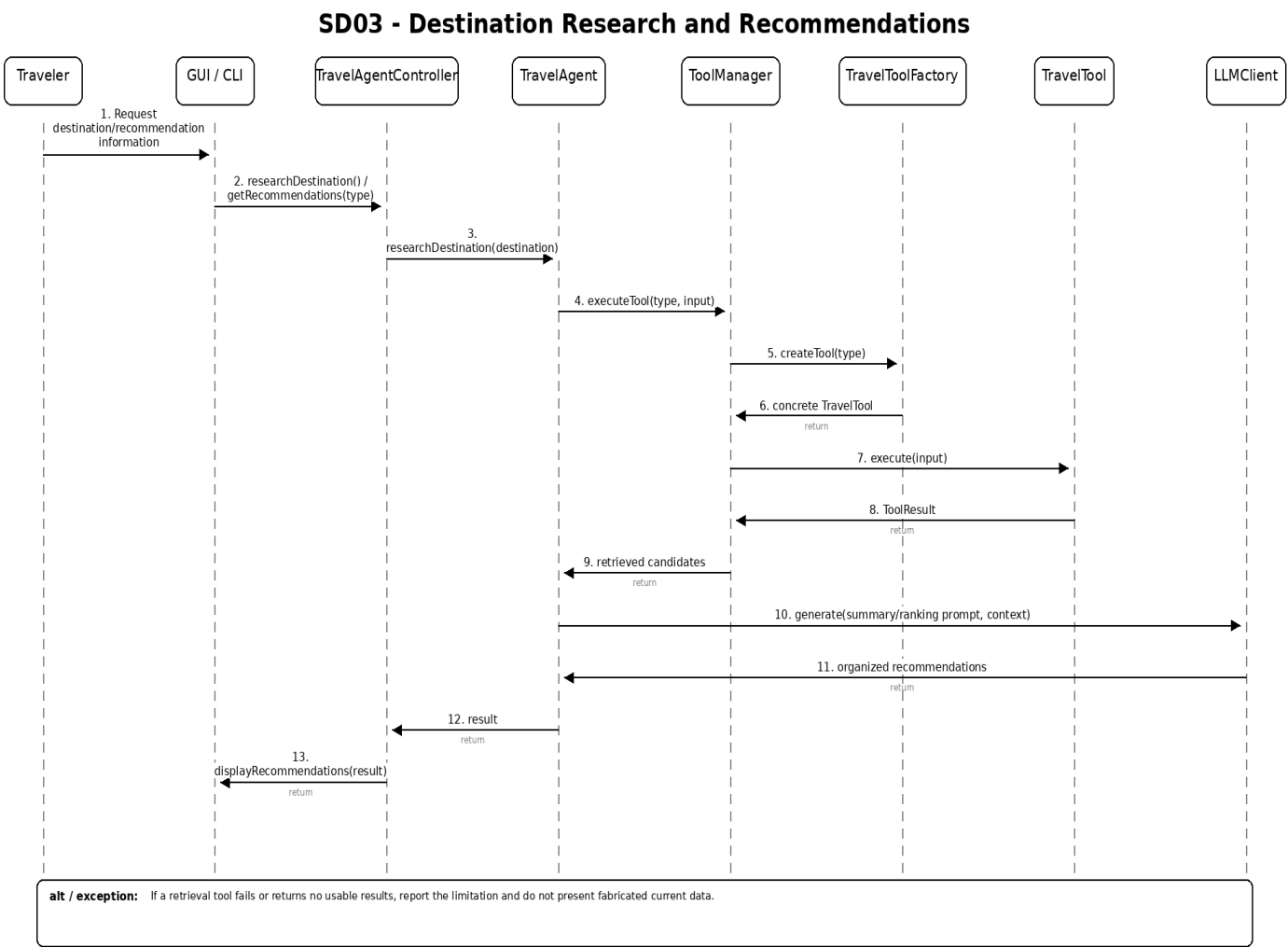
SD02 - Interpret Request and Generate Multi-Day Itinerary

Covered Features: F02, F04



SD03 - Destination Research and Recommendations

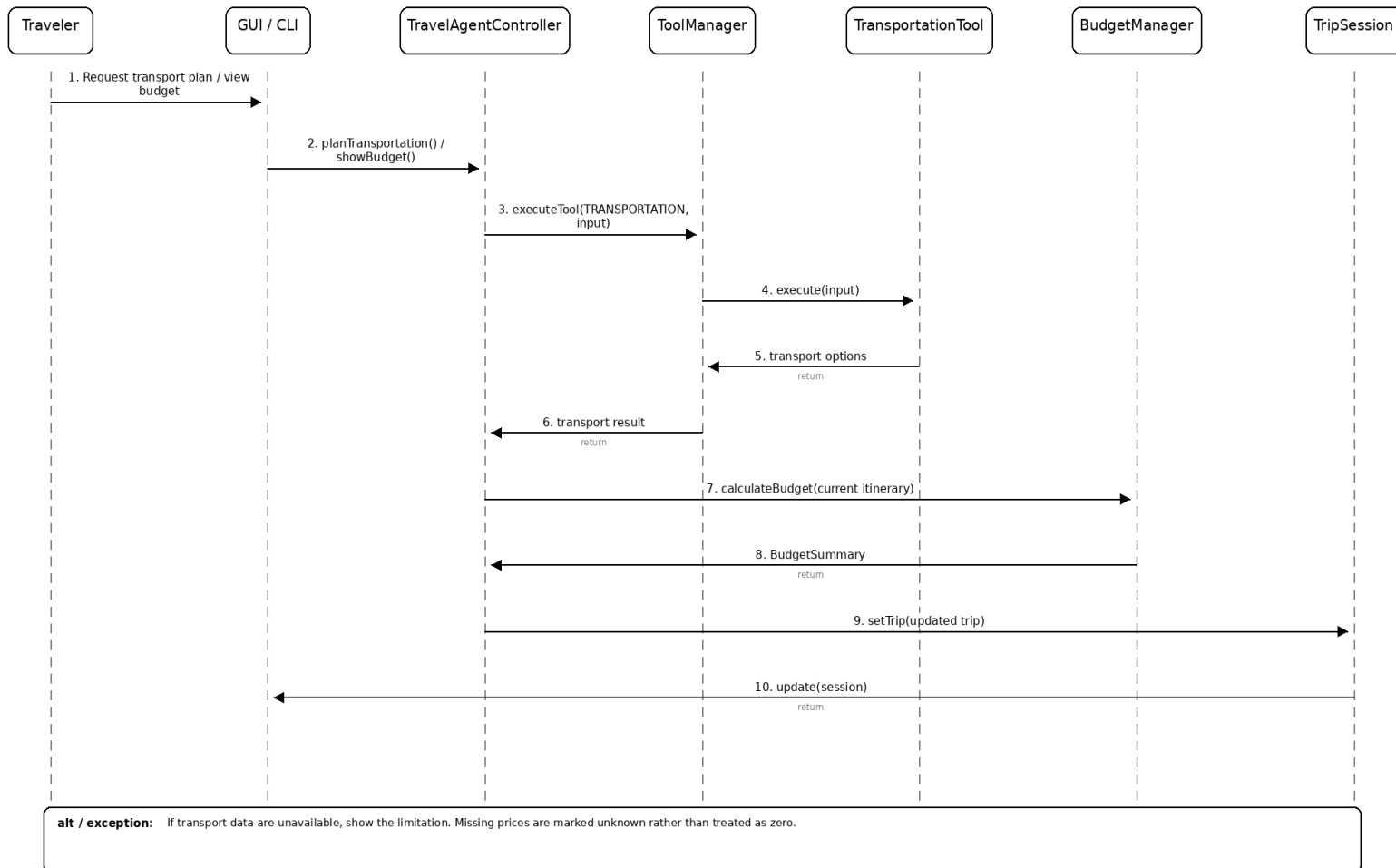
Covered Features: F03, F05, F06, F07



SD04 - Transportation Planning and Budget Tracking

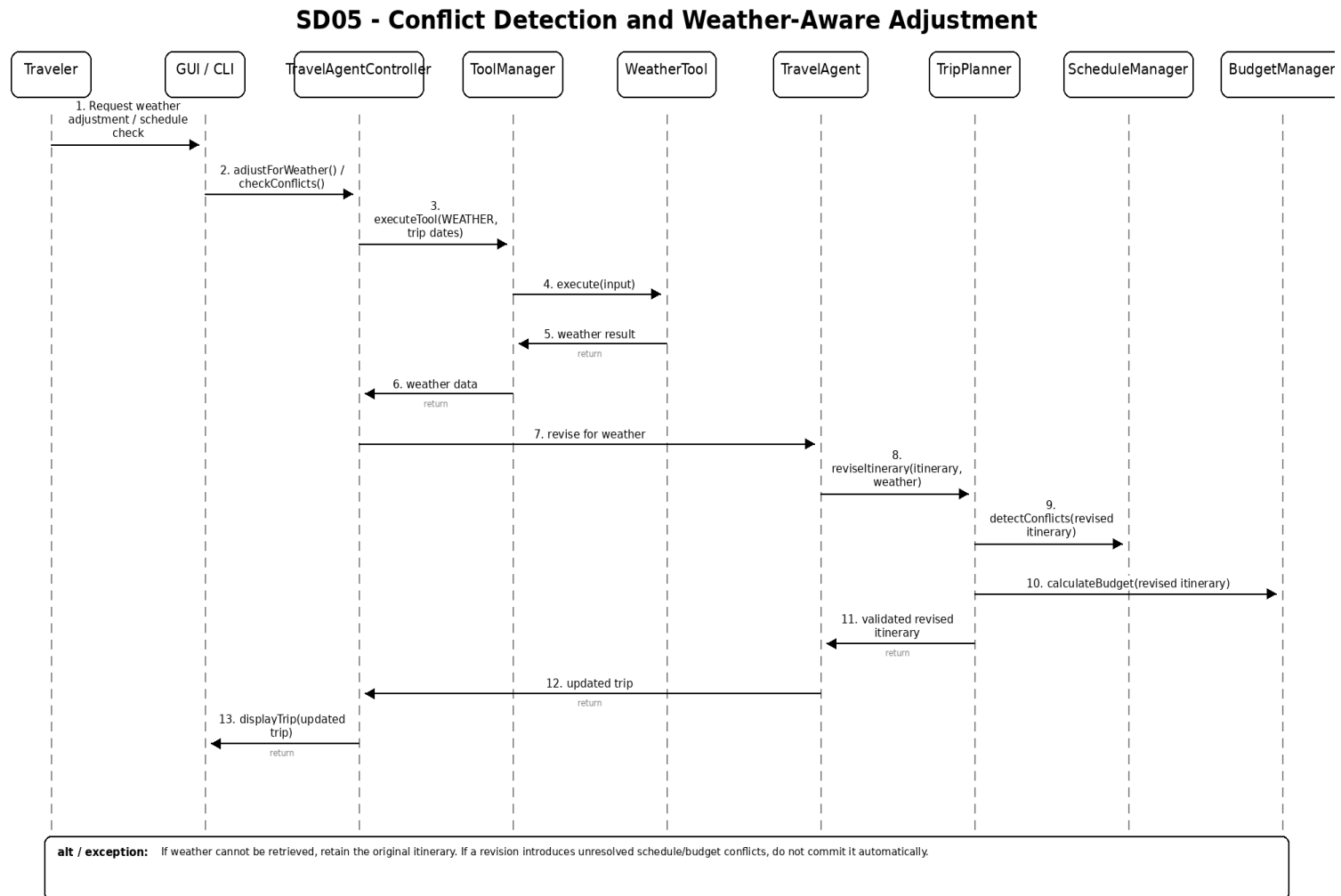
Covered Features: F08, F09

SD04 - Transportation Planning and Budget Tracking



SD05 - Conflict Detection and Weather-Aware Adjustment

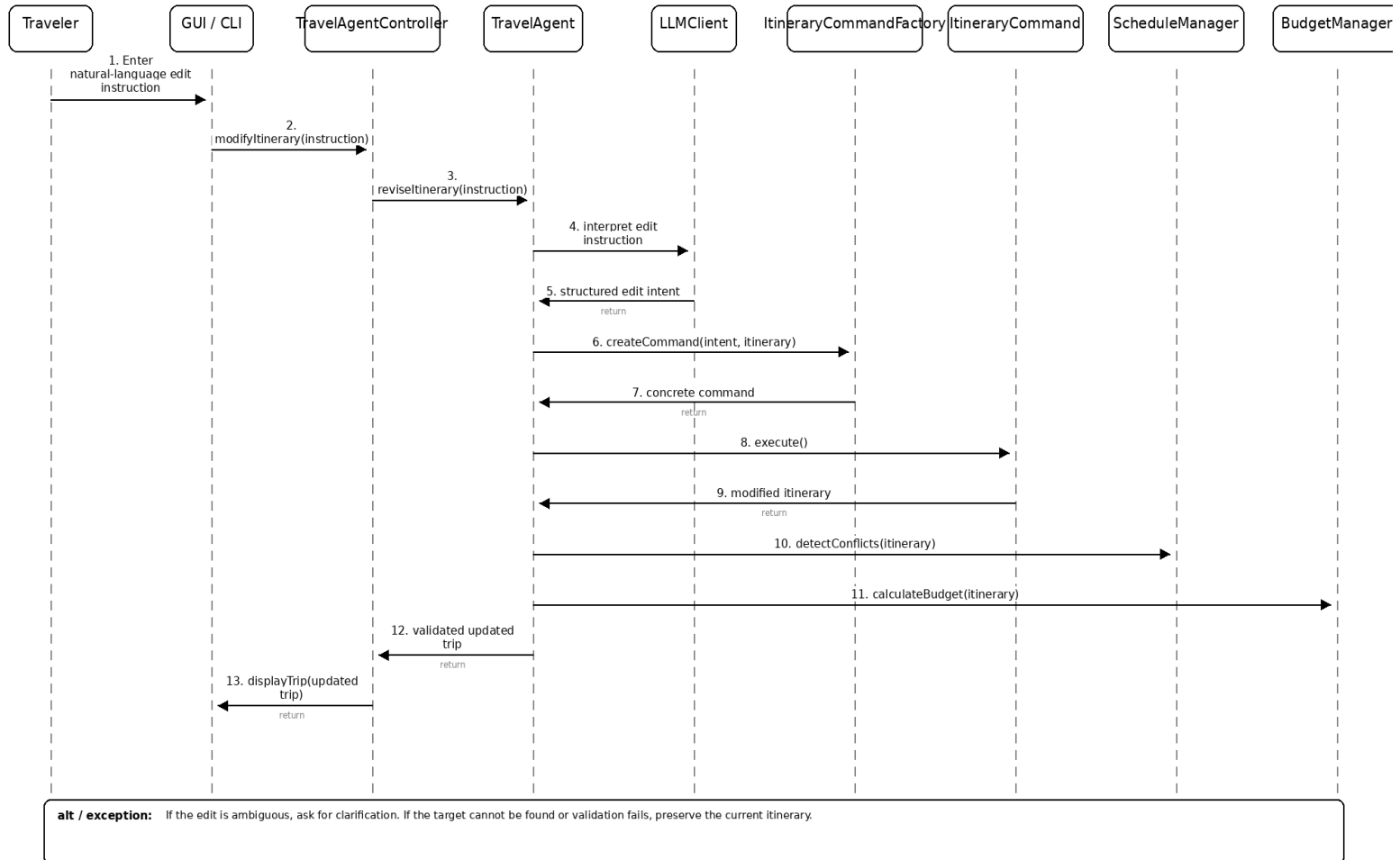
Covered Features: F10, F11



SD06 - Natural-Language Itinerary Editing

Covered Features: F12

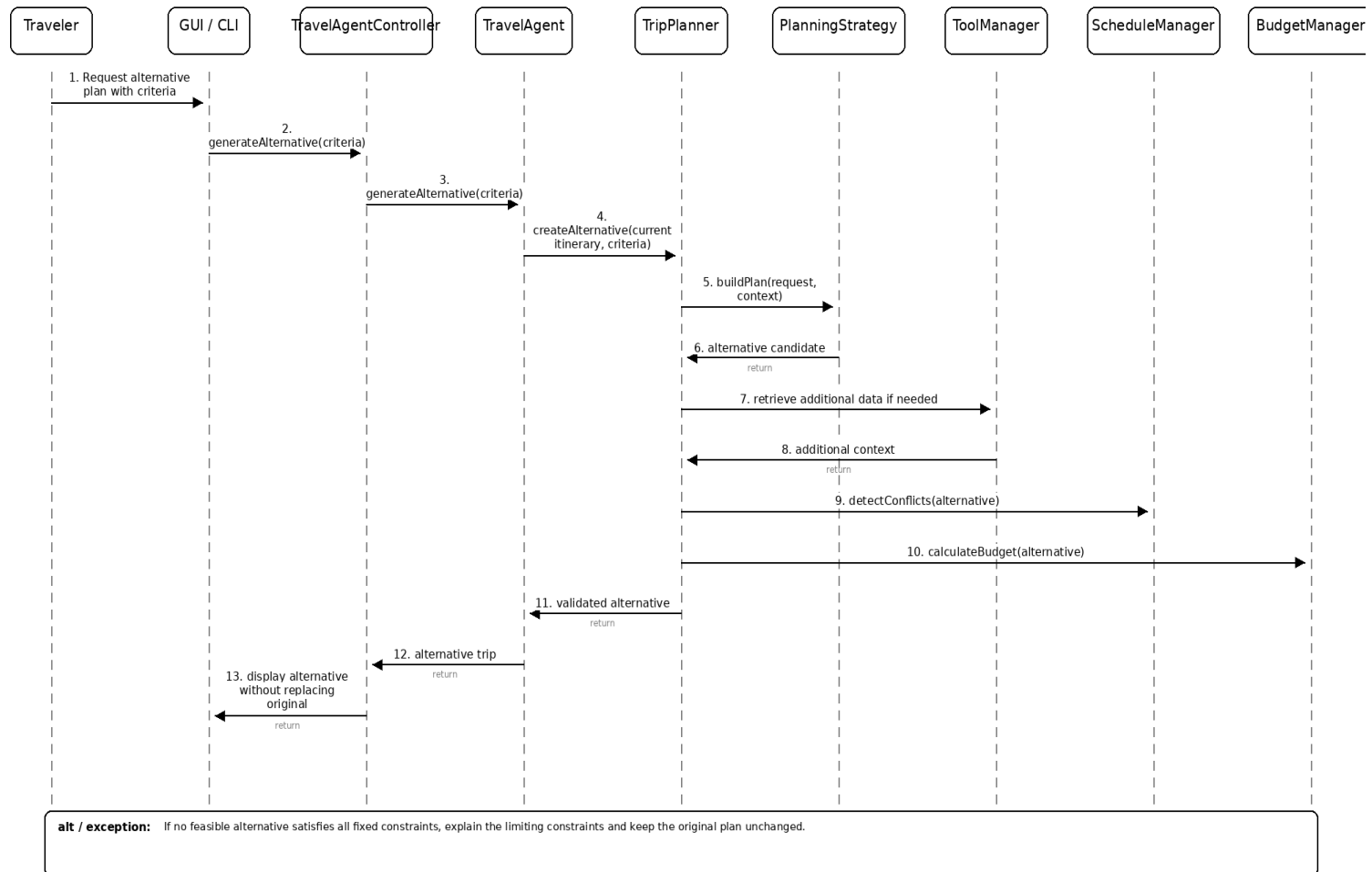
SD06 - Natural-Language Itinerary Editing



SD07 - Generate Alternative Plan

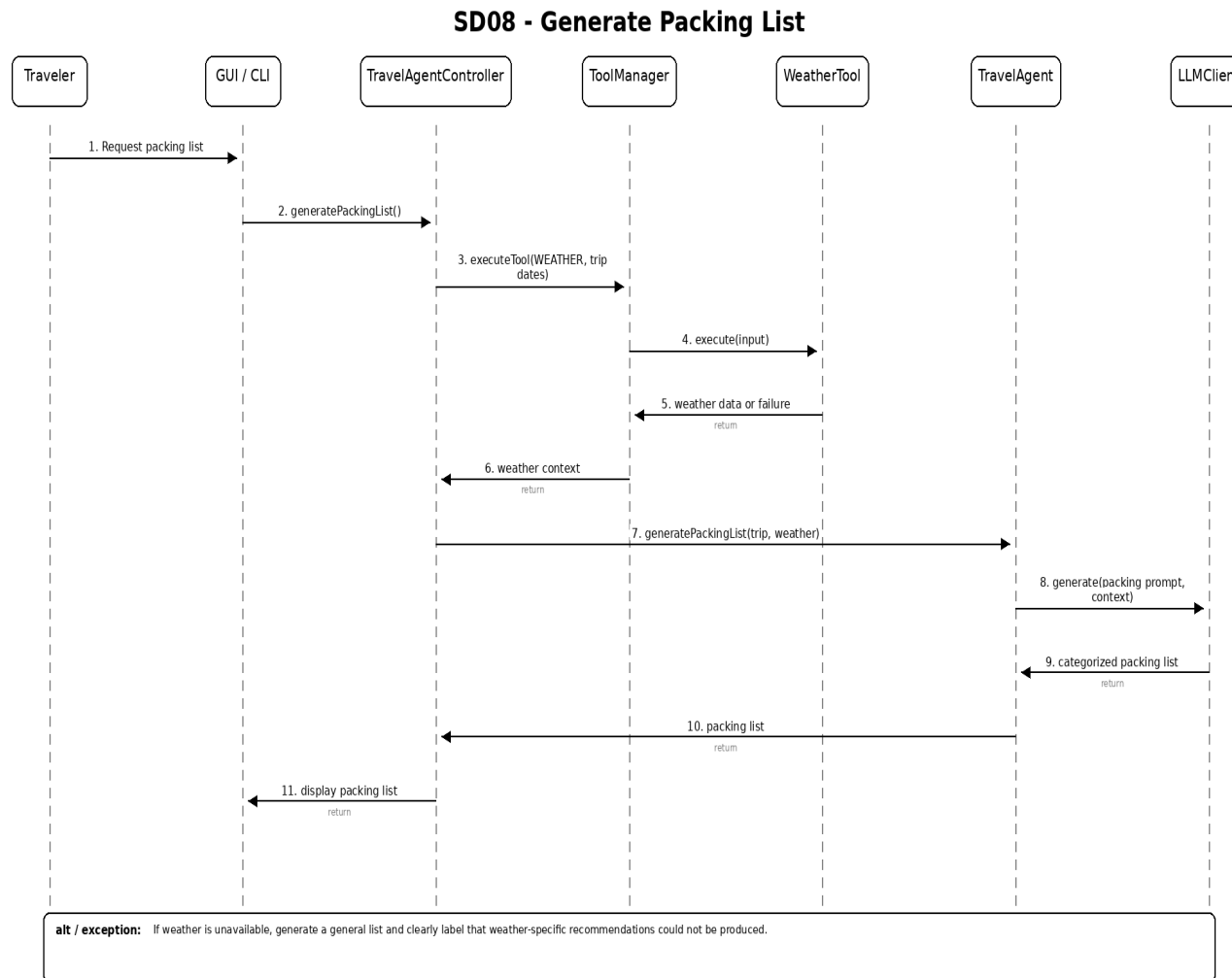
Covered Features: F13

SD07 - Generate Alternative Plan



SD08 - Generate Packing List

Covered Features: F14



The diagrams group closely related features when they share the same runtime interaction structure. All class and method names are consistent with the static design.

8. Feature-to-Design Traceability Table

Feature	Description	Type	Use Case	Classes	Key Methods	Sequence	Patterns
F01	Profile/preferences	Hybrid	UC01	TravelPlannerGUI, CommandLineInterface, TravelAgentController, TravelProfileManager, MemoryManager, TripSession	updateProfile(), validateProfile(), storePreference(), notifyObservers()	SD01	MVC, Facade, Observer
F02	Natural-language request	Hybrid	UC02	GUI/CLI, TravelAgentController, TravelAgent, LLMClient, TripRequest	planTrip(), interpretRequest(), extractTripRequest()	SD02	MVC, Facade
F03	Destination research	Hybrid	UC03	TravelAgentController, TravelAgent, ToolManager, TravelToolFactory, DestinationSearchTool, LLMClient	researchDestination(), executeTool(), createTool(), execute(), generate()	SD03	Facade, Factory Method
F04	Multi-day itinerary	Hybrid	UC04	TravelAgentController, TravelAgent, TripPlanner, PlanningStrategy, ToolManager, LLMClient, ScheduleManager, BudgetManager, TripSession	planTrip(), createItinerary(), buildPlan(), detectConflicts(), calculateBudget(), setTrip()	SD02	Facade, Strategy, Factory Method, Observer
F05	Attraction recommendations	Hybrid	UC05	TravelAgent, ToolManager, TravelToolFactory, AttractionSearchTool, LLMClient	getRecommendations(), executeTool(), createTool(), execute(), generate()	SD03	Facade, Factory Method
F06	Restaurant recommendations	Hybrid	UC06	TravelAgent, ToolManager, TravelToolFactory, RestaurantSearchTool, LLMClient	getRecommendations(), executeTool(), createTool(), execute(), generate()	SD03	Facade, Factory Method
F07	Accommodation planning	Hybrid	UC07	TravelAgent, ToolManager, TravelToolFactory, AccommodationSearchTool, LLMClient	getRecommendations(), executeTool(), createTool(), execute(), generate()	SD03	Facade, Factory Method
F08	Transportation planning	Hybrid	UC08	TravelAgentController, ToolManager, TravelToolFactory, TransportationTool, BudgetManager	planTransportation(), executeTool(), createTool(), execute(), calculateBudget()	SD04	Facade, Factory Method
F09	Budget estimation/tracking	Hybrid	UC09	TravelAgentController, BudgetManager, BudgetSummary, TripSession	showBudget(), calculateBudget(), isWithinBudget(), notifyObservers()	SD04	Facade, Observer
F10	Schedule conflict detection	Hybrid	UC10	TravelAgentController, ScheduleManager, Itinerary	checkConflicts(), detectConflicts(), validateTravelTime()	SD05	Facade
F11	Weather-aware adjustment	Hybrid	UC11	TravelAgentController, ToolManager, WeatherTool, TravelAgent, TripPlanner, ScheduleManager, BudgetManager	adjustForWeather(), executeTool(), revisetItinerary(), detectConflicts(), calculateBudget()	SD05	Facade, Factory Method, Strategy
F12	Natural-language itinerary editing	Hybrid	UC12	TravelAgentController, TravelAgent, LLMClient, ItineraryCommandFactory, ItineraryCommand, ScheduleManager, BudgetManager	modifyItinerary(), revisetItinerary(), createCommand(), execute(), undo(), detectConflicts(), calculateBudget()	SD06	Facade, Command
F13	Alternative plan generation	Hybrid	UC13	TravelAgentController, TravelAgent, TripPlanner, PlanningStrategy, ToolManager, ScheduleManager, BudgetManager	generateAlternative(), createAlternative(), buildPlan(), executeTool(), detectConflicts(), calculateBudget()	SD07	Facade, Strategy, Factory Method
F14	Packing list generation	Hybrid	UC14	TravelAgentController, ToolManager, WeatherTool, TravelAgent, LLMClient	generatePackingList(), executeTool(), generate()	SD08	Facade, Factory Method

9. Feature Realization Explanations

F01 - User Travel Profile and Preferences

``TravelPlannerGUI`` or ``CommandLineInterface`` sends the new ``TravelProfile`` to ``TravelAgentController``. The controller delegates validation and update to ``TravelProfileManager``, stores reusable values through ``MemoryManager``, and updates ``TripSession``. Because the views observe the session, the active interface reflects the saved profile without the domain model depending on UI classes.

F02 - Natural-Language Trip Request

The view calls ``TravelAgentController.planTrip(text)``. The controller delegates interpretation to ``TravelAgent.interpretRequest(text)``, which uses ``LLMClient.extractTripRequest(text)``. The returned structured values are validated before planning. Missing essential values lead to a clarification response rather than an incomplete ``TripRequest``.

F03 - Destination Research

``TravelAgentController.researchDestination()`` delegates to ``TravelAgent``. ``ToolManager.executeTool()`` obtains a ``DestinationSearchTool`` through ``TravelToolFactory.createTool()``. The tool returns retrieved data, and ``LLMClient.generate()`` organizes it into a destination summary used by the traveler and later planning steps.

F04 - Multi-Day Itinerary Generation

``TravelAgentController.planTrip()`` calls ``TravelAgent.planTrip()``, which delegates itinerary creation to ``TripPlanner.createItinerary()``. ``TripPlanner`` selects a ``PlanningStrategy``, obtains needed context through ``ToolManager``, and uses the LLM through the agent. ``ScheduleManager.detectConflicts()`` and ``BudgetManager.calculateBudget()`` validate the proposal. The final ``Trip`` is placed in ``TripSession``, notifying the GUI/CLI.

F05 - Attraction Recommendations

The controller requests attraction recommendations. ``ToolManager`` creates/uses ``AttractionSearchTool``, whose candidates are returned to ``TravelAgent``. The agent combines retrieved data with ``TravelProfile`` and itinerary context and uses ``LLMClient.generate()`` to rank/explain suitable options.

F06 - Restaurant Recommendations

Restaurant recommendations follow the same tool workflow but use ``RestaurantSearchTool``. Deterministic filtering rejects options incompatible with required dietary constraints before the AI organizes the remaining candidates.

F07 - Accommodation Planning

Accommodation planning uses ``AccommodationSearchTool`` to retrieve candidate stays. The agent evaluates candidates against travel dates, budget, preferred area/type, and current itinerary context before returning suitable options.

F08 - Transportation Planning

The controller supplies origin/destination and trip context to ``ToolManager``, which invokes ``TransportationTool``. The result is evaluated against timing and traveler preferences. Selected cost estimates can then be incorporated into ``BudgetManager.calculateBudget()``.

F09 - Budget Estimation and Tracking

``BudgetManager.calculateBudget()`` gathers estimated costs from itinerary activities and trip components and produces a ``BudgetSummary``. ``isWithinBudget()`` compares the total against the user limit. If over budget, the AI may propose substitutions, but all arithmetic remains deterministic.

F10 - Schedule Conflict Detection

``ScheduleManager.detectConflicts()`` sorts/compares itinerary activities and uses available travel-time information through ``validateTravelTime()``. It returns explicit conflicts. The AI may suggest resolutions, but the conflict decision itself is deterministic where the necessary data exist.

F11 - Weather-Aware Itinerary Adjustment

The controller requests weather data through ``ToolManager`` and ``WeatherTool``. ``TravelAgent`` passes the weather context to ``TripPlanner.reviseItinerary()``. The revised plan is accepted only after ``ScheduleManager`` and ``BudgetManager`` revalidate it.

F12 - Natural-Language Itinerary Editing

``TravelAgent`` uses ``LLMClient`` to convert the edit instruction into a structured intent.

``ItineraryCommandFactory.createCommand()`` returns a concrete ``ItineraryCommand`` such as ``MoveActivityCommand`` or ``ReplaceActivityCommand``. After ``execute()``, schedule and budget validation run before the changed trip becomes current state. The command abstraction also supports ``undo()``.

F13 - Alternative Plan Generation

``TripPlanner.createAlternative()`` selects an appropriate ``PlanningStrategy`` according to criteria such as cheaper, slower paced, or more activity-focused. It may retrieve additional data through ``ToolManager``, then validates the alternative with ``ScheduleManager`` and ``BudgetManager``. The alternative is shown separately so the original is preserved until the traveler chooses.

F14 - Packing List Generation

The controller gathers the current trip and requests weather through ``ToolManager`` when available. ``TravelAgent`` passes trip duration, activities, destination, and weather context to ``LLMClient.generate()``. The result is returned as a categorized packing list; if weather is unavailable, the list is clearly labeled as general rather than forecast-specific.

10. Consistency and Stage-1 Design Decisions

The same feature IDs, use-case IDs, class names, method names, sequence-diagram IDs, and design-pattern assignments are used throughout this report. This is intentional so that each proposed feature can be traced from the user-visible requirement to the static structure and runtime interaction.

The current design may evolve during Stage 2 if implementation constraints require changes. Any significant deviation should be documented with the original design, the implemented change, and the reason for the change.

11. Stage-1 Requirement Checklist

- GUI for major functionality: Yes
- CLI for major functionality: Yes
- At least 10 meaningful features: Yes - 14
- At least 5 meaningful design patterns: Yes - 6
- At least one AI/LLM model: Yes
- Meaningful agent behavior (reasoning, planning, retrieval, memory, tool use, multi-step execution): Yes
- UML class diagram: Included
- UML use-case diagram: Included
- Sequence diagrams covering all important behaviors: Included - 8
- Detailed use-case descriptions: Included - 14
- Feature-to-design traceability: Included
- Per-feature realization explanation: Included